# NEURAL-BRIDGE MAS

Technical Infrastructure Manual v1.1

**Subject:** Latent-First Agent Orchestration & vLLM Implementation

**Status:** Engineering Release

**Date:** May 2026

# 1. Executive Summary & Strategic Vision

The **Neural-Bridge MAS** project replaces symbolic text-based communication with direct **Latent State Synchronization**. By utilizing high-density neural vectors, we bypass the "Token Wall" that limits traditional multi-agent systems to 3-4 agents. This manual outlines the production implementation using **vLLM** for raw tensor access and **Differential Privacy (DP)** for neural security.

> **Goal:** Achieve 90%+ token compression and support for 10+ concurrent agents by sharing a **Global K-V Cache** and communicating via **Interlat Tensors**.

# 2. The vLLM Neural Backbone

To extract the ***Last Hidden State (LHS)***, agents must interface with a local vLLM inference engine. Unlike closed APIs, vLLM allows direct access to the model's internal activations.

## 2.1 Tensor Extraction Protocol

Agents utilize a "Neural Hook" within the `model_executor`. The signal is captured at the final layer normalized output, prior to the unembedding head.

```
# Extraction Logic
def get_neural_signal(self, input_ids):
    outputs = self.model.forward(input_ids, return_dict=True)
    # Capture the last hidden state of the last token
    hidden_states = outputs.hidden_states[-1][:, -1, :]
    return hidden_states.detach().cpu().numpy()
```

## 2.2 PagedAttention Alignment

To facilitate simultaneous collaboration, we leverage **PagedAttention**. This allows **Agent A** to "pin" a document's K-V cache blocks so **Agent B** can reference them without re-processing, reducing latency by $O(n)$.

# 3. Differential Privacy: The Inversion Shield

Neural vectors are high-entropy representations that can be "inverted" to reconstruct input data. To prevent this, we implement **Gaussian Differential Privacy**.

## 3.1 Mathematical Implementation

We inject noise $\eta$ into the vector $V$ such that $V' = V + \eta$. The noise is drawn from a normal distribution $N(0, \sigma^2)$.

$$\sigma = (S \sqrt{(2 \ln(1.25/\delta))}) / \varepsilon$$

Where $S$ is the sensitivity (L2 norm), $\varepsilon$ is the privacy budget, and $\delta$ is the probability of privacy failure.

| Epsilon (ε) | Security Level | Logic Fidelity | Drift Target |
|---|---|---|---|
| 0.01 | Maximum | Moderate | 0.10 |
| 0.10 | Standard | High | 0.04 |
| 1.00 | Minimal | Lossless | 0.01 |

# 4. Cross-Model Latent Alignment (Procrustes SVD)

Because **Claude** and **Llama** exist in different vector spaces, we apply **Orthogonal Procrustes Alignment** to create a universal **Neural Bridge**.

## 4.1 The Alignment Algorithm

During the Bootstrapping phase, we find a transformation matrix $W$ that minimizes the distance between the two spaces:

$$min_W \| AW - B \|_F \text{ subject to } W^T W = I$$

This is solved via **Singular Value Decomposition (SVD)**:

```
# Procrustes Alignment
def compute_bridge(A, B):
    M = B.T @ A
    U, S, Vt = np.linalg.svd(M)
    W = Vt.T @ U.T
    return W
```

# 5. SANEmerg & Recovery Protocols

## 5.1 Semantic Importance Filtering

The **SANEmerg Filter** acts as a "low-pass" filter for logic. It identifies dimensions with low variance across the project history and prunes them, leaving only the "Logic Signal."

## 5.2 The 0.12 Drift Fallback

If the **Cosine Distance** between a received signal and the predicted state exceeds **0.12**, the agent must immediately initiate a **Resync Sequence**:

> **Protocol:** RESYNC_RECOVERY
> **Action:** Re-emit last state in "Telegraph English" (TE).
> **Example:** `@ST: LOGIC_DRIFT | !ERR: LATENT_CORRUPTION | RESTART_SYNC`

## 6. Performance Benchmarks (Target)

| Metric | Standard MAS (Text) | Neural-Bridge (v1.1) | Improvement |
| --- | --- | --- | --- |
| Tokens per Task | 12,500 | 850 | ~93% Reduction |
| Agent Capacity | 3-4 Agents | 12-15 Agents | 4x Scalability |
| Logic Accuracy | 82% | 91% | +11% (No Prose Noise) |

---